



Security Assessment

Final Report



MLS Wallet

May 2026

Prepared for Den

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	13
Severity Matrix.....	13
Detailed Findings.....	14
Medium Severity Issues.....	16
M-01: Recovery addresses cannot be revoked once set.....	16
Low Severity Issues.....	17
L-01: ERC-1271 caller accepts oversized return values via lax length check in `_isValidERC1271SignatureNow`	17
L-02: `BatchedTransaction.execute` violates `memory-safe-assembly` by writing sub-call calldata at offset `0x00`	19
Informational Severity Issues.....	21
I-01: Guardian recovery timelock duration is silently discarded when recovery address is zero in `initialize`	21
I-02: `isInitiatorAuthorized` performs a redundant `isGroup` lookup before `isGroupMember`	23
I-03: `RateLimitExceeded` error is reused for an unrelated revert case.....	25
I-04: `_extractContractInnerSignature` writes up to 31 stale bytes past the declared signature length..	26
I-05: `OperationType.AccountTransactionRejection` is dead code, never produced by any flow.....	28
I-06: `dataLength` field uses inconsistent notation in the transaction encoding diagram.....	29
I-07: `BatchedTransaction` discards sub-transaction revert data on failure.....	30
Disclaimer.....	31
About Certora.....	31

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Final Commit Hash	Platform
MLS Wallet	https://github.com/OnChainDen/mls-contracts/	91d2b29	84ea748	EVM

Project Overview

This document describes the security review of **MLS Wallet**. The work was undertaken from **March 24th, 2026** to **May 4th, 2026**.

The following contract list is included in our scope:

src/**/*

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team identified the findings listed on the following page.

Protocol Overview

MLS Wallet is a non-custodial smart-contract wallet for organizations, where every transaction is gated by three independent security layers that must all approve before any state changes onchain take place. The signing client, accessible via the user's phone or Den's developer SDK, validates the request against the organization's policies, an offchain Guardian service independently verifies and co-signs as a separate authority, and the onchain Organization contract enforces the same rules one final time, so an attacker would need to compromise all three layers simultaneously to move a single dollar. The policies themselves are "if-then" rules (who can spend what, to where, up to which limit, with which approvals) stored as a Merkle tree, which lets an organization carry hundreds of fine-grained rules without the gas cost scaling with

policy count. Beyond moving funds, organizations can sign ERC-1271 messages for protocols like Permit2 and CoW, run admin operations behind threshold signatures, configure rate limits and per-initiator spending caps, and rely on two timelocked recovery paths if the Guardian is ever compromised or unavailable. Accounts and Organizations both deploy via CREATE2 against an implementation whitelist that Den controls, so the same organization keeps the same address across every EVM-equivalent chain while still being protected from upgrades to unapproved code.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

System Overview

Den Wallet is a multi-signer EVM smart-wallet platform. An **Organization** (diamond-style facade, EIP-7201 storage) owns a fleet of **Account** BeaconProxies whose beacon is the Organization.

The Organization holds no funds – value lives in the Accounts, and **AccountImplementation.executeTransaction** is gated by **onlyOrganization**. **OrganizationFactory** CREATE2-deploys organizations to the same address on every chain; the per-chain Organization is the EIP-712 **verifyingContract**.

The off-chain Den service is the **Guardian**: a Safe multisig fronted by **SafeExecutorModule**, whose immutable **AUTHORIZED_EXECUTOR** EOA submits on-chain. Most state-changing functions are gated by **onlyGuardian**. The Guardian bundles signatures and submits them, cannot self-approve, and gatekeeps every flow except recovery.

Three signing flows hit the Organization.

- **(1) Account transactions:** initiator + optional reviewer signatures over **InitiateAccountTransaction** / **ReviewAccountTransaction** drive **executeAccountTransaction**, which checks a policy via Merkle proof against **policiesRoot**, runs rate limits, then calls the Account.
- **(2) ERC-1271:** **Account.isValidSignature** static calls **Organization.isValidSignatureForAccount**; **LibOrganizationAccountSignature** dispatches on a 1-byte prefix to recovery (**0x00**) or policy (**0x01**).
- **(3) Admin/member/group/policy/guardian/recovery operations:** admin-signed via **LibOrganizationAdminOperationTimelock**, Guardian-submitted.

Replay protection: an EIP-712 domain binding **chainId** + **verifyingContract**, a redundant **chainId** inside every struct, and a per-org **usedNonces** map keyed on **keccak256(address(this), opType, keccak256(opData), salt)**. Nonces are consumed before the inner call; a revert rolls them back. Recovery runs as two parallel tracks with their own addresses and timelocks: **txRecovery**

(bypasses Guardian + policy for ERC-1271 and account-tx recovery) and `guardianRecovery` (replaces the Guardian).

Threat Surface

Actors – Who Can Do What

- **Guardian** (`LibOrganizationGuardianStorage.guardian`): sole submitter for `executeAccountTransaction`, `rejectAccountTransaction`, admin/member/group/policy operations, account deploy + implementation rotation, normal-track guardian rotation. Cannot self-approve.
- **Authorized Executor EOA** (`SafeExecutorModule.AUTHORIZED_EXECUTOR`, immutable): only caller of `executeOnBehalf`; pays gas for Guardian transactions.
- **Admin**: signs admin operations under timelock + threshold (`LibOrganizationAdminOperationTimelock`).
- **Member / Group member**: initiator/approver of account transactions (`isMember`, `isGroupMember[groupId]`).
- **Tx recovery address**: when enabled, drives `executeRecoveryAccountTransaction` + recovery ERC-1271, **bypassing Guardian and policy**.
- **Guardian recovery address**: independent guardian-rotation track the active Guardian cannot block.
- **Account** (contract-only): callable only by its parent Organization, its only trusted ERC-1271 oracle.

Assets – What Has Value or Security Importance

- **Account balances**: Accounts hold all funds; `Account.executeTransaction` is the only spend path.
- `policiesRoot`: single Merkle root authorizing every spend.
- `usedNonces`: salt-based replay defense.

- **Guardian state** (`guardian`, `pendingGuardian`, `pendingGuardianUpdateTimestamp`, `isGuardianUpdateReadyForAcceptance`).
- **Recovery state** (`txRecovery`, `guardianRecovery`: address, timelock, enable flag, pending state).
- **Membership maps** (`isMember`, `isGroup`, `isGroupMember`, `isAdmin`, `votingThreshold`).
- **Rate-limit usage** (`policyUsage[usageKey][timeWindow]`).
- **accountImplementation**: beacon target reaching every Account.

Actions – Impactful State Transitions and Value Computations

- **executeAccountTransaction**: consumes nonce, runs `LibOrganizationPolicy.isTransactionAllowedByPolicy`, `LibPolicyRateLimits.checkAndUpdateRateLimit`, then `Account.executeTransaction`.
- **isValidSignatureForAccount**: dispatches recovery (`0x00`) or policy-based (`0x01`) ERC-1271.
- **executeRecoveryAccountTransaction**: Guardian-less, policy-less spend gated only by tx-recovery enablement + recovery-address signature.
- **Guardian rotation**: normal + parallel recovery track (`LibOrganizationGuardianRecovery`); last-to-accept wins.
- **Recovery setup**: `initiate*/finalize*RecoveryUpdate`, `enableTxRecovery`.
- **Policy/member/group/admin updates**: `setPolicies` (overwrites `policiesRoot`), `modifyMembers`, `modifyGroups`, `modifyAdmins`.
- **Account deployment / implementation rotation**: `deployAccount`, `setAccountImplementation`.
- **Approval validation** (`LibPolicyApproval.areApprovalsValid`): packed hybrid signature blob, `SignatureUtils.tryRecoverSignerAtOffset`, strict-ascending order, threshold count.
- **Calldata parsing** (`LibPolicyParameterConstraints`, `LibPolicyDestination`, `LibPolicyTokenTransfer`, `BytesUtils`): binds signed transaction to policy.

Core Threat Model

The threats cluster around replay integrity, policy correctness (proofs, approvers, rate limits, calldata parsing), Guardian compromise (unilateral execution + recovery suppression), and liveness (executor gas, ERC-1271 DoS).

Replay of Signatures by the Guardian

A Guardian holding a signed payload could execute an operation twice, or replay it on the same-address Organization on another chain.

Mitigation: Every struct in `LibOrganizationEIP712` embeds `chainId`, and the domain binds `chainId` + `verifyingContract`. Same-chain replay is blocked by `LibOrganizationSignatures.validateAndConsumeNonceOrRevert`, which sets `usedNonces[nonce]` keyed on `keccak256(address(this), opType, keccak256(opData), salt)` before the inner call.

Organization Spoofing

A malicious Organization could call into an Account it doesn't own, or relay another org's signed payload as its own.

Mitigation: `AccountImplementation.executeTransaction` is gated by `onlyOrganization`; the trusted Organization is fixed at deploy in `LibAccountOrganizationAddressStorage`. Signed structs include both `organization` and `account`, so a signature for org A cannot be lifted into org B – both the struct hash and the domain's `verifyingContract` differ.

Using Malicious Proofs to Validate Against the Merkle Root

Forged proofs would let an attacker invent policies, source accounts, destinations, or `OneOf` values admins never authorized.

Mitigation: `LibOrganizationPolicy.isTransactionAllowedByPolicy` verifies the leaf via `MerkleProof.verify(policyProof, policiesRoot, hashPolicy(policyId, policy))`; `LibPolicyDestination` and `LibPolicyParameterConstraints` apply the same pattern to per-policy destination and `OneOf` roots. `policiesRoot` is writable only via admin-authorized `setPolicies` — gated by `onlyGuardian` submission plus a threshold admin-signature check (`OperationType.ModifyPolicies`); `hashPolicy` prepends typed data so leaves cannot collide with internal nodes.

Policy Performing Erroneous Validations

A transaction the policy meant to forbid (wrong destination, token, amount, selector, initiator) would be accepted, breaking the spend boundary.

Mitigation: `isTransactionAllowedByPolicy` chains every check – Merkle existence → source-account (`anySourceAccount` or proof) → `LibPolicyInitiator.isInitiatorAuthorized` → tx-type → `LibPolicyDestination.isDestinationAllowedByPolicy` (extracts the ERC-20 recipient when relevant) → `LibPolicyTokenTransfer` amount cap → `LibPolicyParameterConstraints.areParametersAllowedByConstraints`. Any failure reverts before the inner call.

Bypassing the Policy Rate Limits

A member could re-use one policy to drain past `timeIntervalLimit` by toggling time, scope, or accounting.

Mitigation: `LibPolicyRateLimits.checkAndUpdateRateLimit` derives `usageKey = keccak256(policyId + scoped entities)` from `limitScope` flags (initiator/source/destination), computes `timeWindow = (block.timestamp - anchorTimestamp) / (hours * 3600)`, and **fails closed** when `block.timestamp < anchorTimestamp` or `hours == 0`. Enforces `currentUsage + usageAmount <= timeIntervalLimit` before writing storage.

Compromised Guardian Unilaterally Executing Transactions

A malicious Guardian could move user funds with no user signature.

Mitigation: Guardian privilege is submission-only. Every spend path verifies user EIP-712 signatures via `LibPolicyApproval` / `LibOrganizationTxRecovery.isValidRecoverySignature` against signers the Guardian doesn't control (members, admins, recovery). The Guardian cannot self-approve – it's neither a member nor group member.

Compromised Guardian Preventing Recovery Addresses from Being Set

A Guardian able to veto recovery setup would leave a captured Organization without a fallback mechanism.

Mitigation: Recovery setup is admin-signed; submission is fungible – any honest submitter of an admin-signed payload can land it. `txRecovery` and `guardianRecovery` are **independent tracks** in `LibOrganizationRecoveryStorage`; the recovery-track rotation runs in parallel to the normal track, callable via `onlyGuardianRecoveryAddress` (not the Guardian), with its own pending state and timelock.

Compromised Guardian Preventing Recovery Addresses from Executing Transactions

A compromised guardian rejecting or preventing the execution of a transaction or signature submitted by a recovery address while they should have complete authority.

Mitigation: `executeRecoveryAccountTransaction` and the ERC-1271 recovery branch (`0x00`) are gated by `onlyTxRecoveryAddress` / recovery-signature, not `onlyGuardian`. They skip the policy engine, accepting only an EIP-712 `RecoverySignatureValidation` from `txRecovery.recoveryAddress` with explicit `expirationTimestamp`. While `txRecovery.isEnabled`, the Guardian is out of the path.

Incorrect Parsing of Raw Bytes Across the Codebase

Raw bytes carry authority across calldata parsing, parameter constraints, signatures, Merkle leaves, and Guardian batches. A wrong offset, malformed length, layout confusion, or hash/execution mismatch could approve one operation while executing another.

Mitigation: Policies, `SafeExecutorModule`, `BatchedTransaction` and custom signature format are parsed correctly. They interpret each bytes sequences following the intended format and revert if the bytes sequence is incorrect.

Incomplete Parameter and Signature Validations

Skipped checks (only first N params, sig blob shorter than declared, ignored `expirationTimestamp`) silently widen what the system accepts.

Mitigation: `LibPolicyParameterConstraints.areParametersAllowedByConstraints` iterates the full `ParameterConstraint[]` and reverts on any mismatch; `LibPolicyApproval.areApprovalsValid` walks the entire packed buffer, asserts it is fully consumed, and rejects malformed ERC-1271 entries via the `sigLength` field. Every signed flow checks `block.timestamp <= expirationTimestamp` first.

Exhausting the Authorized Executor EOA Gas

If `AUTHORIZED_EXECUTOR` runs out of ETH, the Guardian cannot submit resulting in a soft DoS for every Organization sharing it.

Mitigation: The EOA is monitored off-chain and the gas limit when executing transaction batches is computed to prevent gas exhaustion.

In addition, the disaster-recovery path does not depend on the shared executor: when transaction recovery is enabled, `executeRecoveryAccountTransaction` is gated by `onlyTxRecoveryAddress` and is submitted directly by the organization's recovery address — not by `AUTHORIZED_EXECUTOR` — so a gas-starved or otherwise unavailable executor cannot prevent an organization from moving funds through recovery.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	1	1	1
Low	2	2	2
Informational	7	5	5
Total	10	8	8

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	Recovery addresses cannot be revoked once set	Medium	Fixed
L-01	ERC-1271 caller accepts oversized return values via lax length check in <code>`_isValidERC1271SignatureNow`</code>	Low	Fixed
L-02	<code>`BatchedTransaction.execute`</code> violates <code>`memory-safe-assembly`</code> by writing sub-call calldata at offset <code>`0x00`</code>	Low	Fixed
I-01	Guardian recovery timelock duration is silently discarded when recovery address is zero in <code>`initialize`</code>	Informational	Acknowledged
I-02	<code>`isInitiatorAuthorized`</code> performs a redundant <code>`isGroup`</code> lookup before <code>`isGroupMember`</code>	Informational	Fixed
I-03	<code>`RateLimitExceeded`</code> error is reused for an unrelated revert case	Informational	Fixed
I-04	<code>`_extractContractInnerSignature`</code> writes up to 31 stale bytes past the declared signature length	Informational	Acknowledged
I-05	<code>`OperationType.AccountTransactionRejection`</code> is dead code, never produced by any flow	Informational	Fixed

I-06	`dataLength` field uses inconsistent notation in the transaction encoding diagram	Informational	Fixed
I-07	`BatchedTransaction` discards sub-transaction revert data on failure	Informational	Fixed

Medium Severity Issues

M-01: Recovery addresses cannot be revoked once set

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: OrganizationGuardianRecoveryBase.sol	Status: Fixed	

Description:

Recovery addresses for transaction and guardian recovery can only be set when un-configured, and there is no mechanism to rotate, cancel, or revoke them once they have been set.

If a recovery address is compromised or its keys are lost, the organization has no way to cancel a pending recovery or revoke the compromised address.

Recommendations:

Implement a mechanism to reset the recovery addresses back to address(0) to effectively revoke them.

Customer Response:

Fixed in [PR#153](#).

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: ERC-1271 caller accepts oversized return values via lax length check in `_isValidERC1271SignatureNow`

Severity: Low	Impact: Low	Likelihood: Low
Files: SignatureUtils.sol#L194	Status: Fixed	

Description:

`_isValidERC1271SignatureNow` validates an ERC-1271 contract signature by `staticcall`-ing `isValidSignature(bytes32,bytes)` on the signer and decoding the first 32 bytes of the returned data:

```
function _isValidERC1271SignatureNow(address signer, bytes32 hash, bytes memory signature)
    internal
    view
    returns (bool)
{
    (bool success, bytes memory result) =
        signer.staticcall(abi.encodeCall(IERC1271.isValidSignature, (hash,
signature)));
    return (success && result.length >= 32 && abi.decode(result, (bytes32)) ==
bytes32(ERC1271_MAGIC_VALUE));
}
```

The length guard uses `result.length >= 32`, which accepts any return blob that is at least 32 bytes. ERC-1271 specifies that `isValidSignature` returns a single `bytes4` magic value, ABI-encoded to exactly 32 bytes. Returning more (or less) than 32 bytes is a non-conforming response. The current check is permissive: a contract that returns 64, 128, or any larger blob whose first 32 bytes match the magic value is still treated as a valid signer.

Recommendations:

Tighten the length check to require exact spec compliance:

```
return (success && result.length == 32 && abi.decode(result, (bytes32)) ==  
bytes32(ERC1271_MAGIC_VALUE));
```

Customer Response:

Fixed in [PR#142](#).

Fix Review:

Fix confirmed.

L-02: `BatchedTransaction.execute`` violates ``memory-safe-assembly`` by writing sub-call calldata at offset `0x00``

Severity: Low	Impact: Low	Likelihood: Low
Files: BatchedTransaction.sol#L57	Status: Fixed	

Description:

`BatchedTransaction.execute` decodes packed transactions and dispatches each sub-call via `call`, copying the sub-call's calldata into memory starting at offset `0x00`:

```
// Copy sub-transaction's calldata to memory scratch space for the call
let data := 0x00
calldatacopy(data, currentOffset, dataLength)

// Execute the call with value=0 (hardcoded for security)
// call(gas, to, value, inOffset, inSize, outOffset, outSize)
let success := call(gas(), to, 0, data, dataLength, 0x00, 0x00)
```

Writing sub-call calldata starting at `0x00` overwrites memory regions the Solidity compiler reserves for its own use:– `0x00–0x3f`: Solidity scratch space, clobbered for any `dataLength > 0`.– `0x40`: free-memory pointer, clobbered for any `dataLength >= 64`.– `0x60`: zero slot (must remain zero), clobbered for any `dataLength >= 96`.

The block is not annotated `assembly ("memory-safe")`, and it cannot be because it does not respect Solidity's memory model.

As-is, the code happens to be safe: the assembly block is the last operation in `execute`, and the contract is intended to be `delegatecall`-ed from a Safe so the polluted memory belongs to the calling frame's `delegatecall` sub-context only.

But the safety relies on two implicit invariants:1. No Solidity-managed memory access follows the loop in this function.2. No future maintainer adds Solidity code below or between iterations of the loop.

Both invariants are easy to break with a one-line change. The compiler is also free, in principle, to apply optimizations (e.g. memory reuse, intermediate result caching) that assume the standard memory layout is intact across this region.

Recommendations:

Use the real free-memory pointer instead of `0x00` so the block respects Solidity's memory model.

```
// Copy sub-transaction's calldata starting at the real free-memory pointer
let data := mload(0x40)
calldatacopy(data, currentOffset, dataLength)

let success := call(gas(), to, 0, data, dataLength, 0x00, 0x00)
```

Customer Response:

Fixed in [PR#141](#).

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Guardian recovery timelock duration is silently discarded when recovery address is zero in `initialize`

Files:

[LibOrganizationInitialization.sol#L80-L85](#)

Description:

`LibOrganizationInitialization.initialize` gates the optional guardian recovery setup on the recovery address only:

```
if (params.guardianRecoveryAddress != address(0)) {  
    LibOrganizationGuardianRecovery.initializeGuardianRecovery({  
        guardianRecoveryAddress: params.guardianRecoveryAddress,  
        guardianRecoveryTimelockDurationSeconds:  
params.guardianRecoveryTimelockDurationSeconds  
    });  
}
```

A caller passing `guardianRecoveryAddress = address(0)` together with a non-zero `guardianRecoveryTimelockDurationSeconds` does not get any feedback – the timelock value is silently dropped, and `initialize` succeeds.

The check is one-sided: address presence implies "set up recovery", but a non-zero timelock duration is a strong indicator the deployer *intended* to enable recovery and the missing address is a configuration mistake.

The same shape applies to the tx/ERC1271 recovery block immediately below (lines 88–92).

Recommendations:

Validate the parameters together, so an inconsistent pair reverts.

Either require both to be zero or both non-zero:

```
if (params.guardianRecoveryAddress != address(0) &&  
params.guardianRecoveryTimelockDurationSeconds != 0) {
```

```
    LibOrganizationGuardianRecovery.initializeGuardianRecovery({  
        guardianRecoveryAddress: params.guardianRecoveryAddress,  
        guardianRecoveryTimelockDurationSeconds:  
params.guardianRecoveryTimelockDurationSeconds  
    });  
}
```

Apply the same symmetry to the tx/ERC1271 recovery block at lines 88–92 for consistency.

Customer Response:

Acknowledged, won't fix. When recovery is actually being configured, the timelock duration is validated, not silently skipped. Both `LibOrganizationGuardianRecovery.initializeGuardianRecovery` and `LibOrganizationTxRecovery.initializeTxRecovery` validate the timelock duration via `TimelockUtils.validateTimelockDurationOrRevert`, which reverts with a specific `InvalidTimelockDuration(provided, min, max)` error whenever the duration falls outside the allowed range (2–30 days). We consider this more informative and useful than the recommended combined check: the dedicated `InvalidTimelockDuration` error reports the offending value together with the valid bounds, giving the deployer a precise, actionable reason for the revert rather than silently dropping the value.

I-02: `isInitiatorAuthorized` performs a redundant `isGroup` lookup before `isGroupMember`

Files:

[LibPolicyInitiator.sol#L48-L50](#)

Description:

`LibPolicyInitiator.isInitiatorAuthorized` checks group existence and membership in two steps:

```
if (initiatorType == ApproverType.Group) {
    uint256 initiatorGroupId = policy.config.initiator.initiatorGroupId;

    // Verify the group exists
    if (!LibOrganizationGroups.isGroup(initiatorGroupId)) {
        return false;
    }

    // Verify the initiator is a member of the group
    return LibOrganizationGroups.isGroupMember(initiatorGroupId,
initiatorAddress);
}
```

`LibOrganizationGroups.isGroupMember` already performs the same `isGroup` check internally before reading the membership bit:

```
function isGroupMember(uint256 groupId, address memberAddress) internal view
returns (bool) {
    LibOrganizationGroupsStorage.Layout storage groupsLayout =
LibOrganizationGroupsStorage.layout();

    if (!groupsLayout.isGroup[groupId]) return false;

    return groupsLayout.isGroupMember[groupId][memberAddress];
}
```

The outer `isGroup` call in `isInitiatorAuthorized` is therefore redundant: the inner check inside `isGroupMember` already returns `false` when the group is missing. Both calls hit the same storage slot, so the duplicate is a pure gas overhead with no behavioral difference.

Recommendations:

Drop the outer `isGroup` call and rely on `isGroupMember`'s internal check:

```
if (initiatorType == ApproverType.Group) {  
    return  
    LibOrganizationGroups.isGroupMember(policy.config.initiator.initiatorGroupId,  
    initiatorAddress);  
}
```

Customer Response:

Fixed in [PR#143](#).

Fix Review:

Fix confirmed.

I-03: `RateLimitExceeded` error is reused for an unrelated revert case

Files:

[LibOrganizationAccountTransaction.sol#L168](#)

Description:

In [LibOrganizationAccountTransaction.sol:168](#), when [rateLimitType](#) holds an unrecognized enum value, the code reverts with [RateLimitExceeded](#) – a fail-closed guard against invalid configuration, not a quota exhaustion event. The two failure modes have different causes and different severities, so conflating them under the same error makes it harder to distinguish between a legitimate rate limit hit and an unexpected configuration problem when investigating a revert.

Recommendations:

Introduce a dedicated error – for example [UnknownRateLimitType\(uint256 policyId\)](#) – and use it in the guard at line 168 instead of [RateLimitExceeded](#).

Customer Response:

Fixed in [PR#140](#).

Fix Review:

Fix confirmed.

I-04: `_extractContractInnerSignature` writes up to 31 stale bytes past the declared signature length

Files:

[SignatureUtils.sol#L334](#)

Description:

`_extractContractInnerSignature` allocates a fresh `bytes` buffer of length `sigLength` and copies the inner ERC-1271 signature into it word-by-word using inline assembly:

```
function _extractContractInnerSignature(bytes memory data, uint256 offset, uint16 sigLength)
    internal
    pure
    returns (bytes memory contractSig)
{
    contractSig = new bytes(sigLength);
    assembly {
        let src := add(add(data, 0x20), add(offset, 23))
        let dst := add(contractSig, 0x20)
        for { let i := 0 } lt(i, sigLength) { i := add(i, 0x20) } {
mstore(add(dst, i), mload(add(src, i))) }
    }
}
```

The loop advances 32 bytes at a time and runs while `i < sigLength`. When `sigLength` is not a multiple of 32, the final iteration still issues a full 32-byte `mstore`, so the last write spills `32 - (sigLength mod 32)` bytes – up to 31 past the declared signature boundary. Those spilled bytes come from the source buffer beyond the inner signature (the next concatenated signature in `signatures`, padding, or the next allocation). This is harmless in the current implementation because consumers of `contractSig` read only `sigLength` bytes. The trailing bytes live in the buffer's padding region, which Solidity normally treats as garbage.

Recommendations:

Mask the trailing bytes after the copy loop, just like implemented in `BytesUtils::sliceRange()`.

Customer Response:

Acknowledged, won't fix. There is no path in which a valid signature fails validation, an invalid signature is accepted, or any data is exfiltrated or injected: every consumer of the returned bytes respects the declared length prefix. Replacing `_extractContractInnerSignature` with `BytesUtils::sliceRange()`, or adding equivalent masking for trailing bytes, would push `OrganizationImplementation` past the bytecode size limit for deployment and would require larger changes to reduce the overall bytecode size.

I-05: `OperationType.AccountTransactionRejection` is dead code, never produced by any flow

Files:

[CommonTypes.sol#L23-L44](#)

Description:

`OperationType` enumerates every operation that can be authorized via signed admin or account-tx flows. The last value, `AccountTransactionRejection`, is declared at `CommonTypes.sol:43` but is never used to identify a real operation anywhere in `src/`.

No code path ever produces an `AccountTransactionRejection` value, so this branch is unreachable from any signed input – the enum value has no semantic role and the guard against it is dead code.

Recommendations:

Delete `AccountTransactionRejection` from `OperationType` and drop the corresponding clause in `OrganizationAdminBase.rejectAdminOperation`.

Customer Response:

Fixed in [PR#139](#).

Fix Review:

Fix confirmed.

I-06: `dataLength` field uses inconsistent notation in the transaction encoding diagram

Files:

[GUARDIAN_PROTECTION.md](#)

Description:

The transaction encoding diagram in [GUARDIAN_PROTECTION.md:173](#) labels the `dataLength` field as `(8B)`, while the two adjacent fields follow a different convention – `to` is labeled `(20 bytes)` and `data` is labeled `(N bytes)`. The abbreviation `8B` is technically unambiguous, but the inconsistency within the same diagram makes the format harder to read and introduces unnecessary friction for auditors and integrators trying to understand the encoding.

Recommendations:

Change `dataLength (8B)` to `dataLength (8 bytes)` to match the notation used by the other fields in the diagram.

Customer Response:

Fixed in [PR#134](#).

Fix Review:

Fix confirmed.

I-07: `BatchedTransaction` discards sub-transaction revert data on failure

Files:

[BatchedTransaction.sol#L66](#)

Description:

In [BatchedTransaction.sol:66](#), when a sub-transaction in the batch fails, the code reverts with `revert(0x00, 0x00)` – discarding the sub-transaction's revert data entirely. The revert reason from the failed call is available via `returndatasize` and `returndatacopy` after a failed `call()`, but the code never captures it. This makes debugging a failed batch significantly harder, as the root cause of the failure is lost and the caller receives no information about which sub-transaction failed or why.

Recommendations:

Replace the empty revert with a bubble-up pattern.

Customer Response:

Fixed in [PR#144](#).

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.